# ECE 527: High Performance Systems Design

## Project 1: Leiserson-Saxe Circuit Retiming Algorithm Implementation & Verification

**Student Name:** Allen Harrison Paul
**Student ID:** 954353816
**Date:** October 28, 2026

## Academic Integrity & Honor Pledge

---

***"On my honor, I have neither given nor received unauthorized aid on this work."***

| | |
|---|---|
| **Signed / Typed Name:** | **Allen Harrison Paul** |
| **Date:** | **October 28, 2026** |

# Project Overview

Retiming is a fundamental structural transformation for synchronous digital networks that relocates flip-flops (registers) across combinational logic blocks without changing the circuit's overall functional topology. My approach to this report was done using modular C implementation of the Leiserson-Saxe retiming algorithm. The software suite parses arbitrary directed circuit netlists, evaluates the all-pairs minimum register matrix $W(u, v)$ and critical combinational delay matrix $D(u, v)$, constructs difference constraint graphs, and employs Bellman-Ford shortest-path routing to determine the minimum feasible clock period $c_{\min}$ and optimal node retiming vector $r(v)$. The implementation is verified across three benchmarks: an in-class reference circuit, a 5-node pipelined datapath, and a multi-loop feedback network.

# Contents

**1 Leiserson-Saxe Retiming Theory** **2**
1.1 Retiming Transformation . . . . . 2
1.2 Matrix Formulations: $W(u, v)$ and $D(u, v)$ . . . . . 2
1.3 System of Difference Constraints . . . . . 2

**2 Software Implementation** **2**

**3 Experimental Results** **3**
3.1 Example 1: In-Class Reference Circuit (IIR Circuit) (`example_class.txt`) . . . 3
3.2 Example 2: 5-Node Pipelined Datapath (`5_node_pipeline.txt`) . . . . . 3
3.3 Example 3: Multi-Loop Network (`multi_loop.txt`) . . . . . 4

**4 Program Execution Logs** **4**

**5 AI Usage and Steps** **5**

**6 Readings** **11**
6.1 Retiming: Theory and Practice - N. Shenoy . . . . . 11
6.2 Retiming for Wire Pipelining in System-on-Chip - H. Zhou . . . . . 13

**7 Conclusion** **15**

# 1 Leiserson-Saxe Retiming Theory

A synchronous circuit is modeled as a finite directed multigraph $G = (V, E, d, w)$, where each vertex $v \in V$ represents a combinational logic block with propagation delay $d(v) > 0$, and each directed edge $e = (u, v) \in E$ represents a interconnect wire weighted with $w(e) \geq 0$ flip-flops.

## 1.1 Retiming Transformation

A retiming operation is an integer vertex mapping $r : V \to \mathbb{Z}$ that shifts $r(v)$ registers from all output edges of node $v$ to all input edges of node $v$. The retimed register count $w_r(e)$ on an edge $e = (u, v)$ is defined by:

$$w_r(e) = w(e) + r(v) - r(u) \tag{1}$$

For a path $p : u \rightsquigarrow v$, the retimed path register count is $w_r(p) = w(p)+r(v)-r(u)$. Note that for any closed cycle $C$, $w_r(C) = w(C)$, proving that retiming preserves cycle register counts.

## 1.2 Matrix Formulations: $W(u, v)$ and $D(u, v)$

To characterize critical paths, we compute two matrices for all pairs $(u, v) \in V \times V$:

1. **Matrix $W(u, v)$:** The minimum register weight along any path $p : u \rightsquigarrow v$:

$$W(u, v) = \min\{w(p) \mid p : u \rightsquigarrow v\} \tag{2}$$

   Computed using the Floyd-Warshall all-pairs shortest path algorithm.

2. **Matrix $D(u, v)$:** The maximum zero-register propagation delay along any path $p : u \rightsquigarrow v$ achieving register weight $W(u, v)$:

$$D(u, v) = \max\{d(p) \mid p : u \rightsquigarrow v \text{ and } w(p) = W(u, v)\} \tag{3}$$

## 1.3 System of Difference Constraints

To achieve a target clock period $c$, the retiming vector $r(v)$ must satisfy two constraint types:

1. **Legality Constraints:** Retimed edge register counts must be non-negative:

$$w_r(e) \geq 0 \implies r(u) - r(v) \leq w(e) \quad \forall e = (u, v) \in E \tag{4}$$

2. **Clock Period Constraints:** Paths with propagation delay $D(u, v) > c$ must contain at least 1 register after retiming:

$$r(u) - r(v) \leq W(u, v) - 1 \quad \forall (u, v) \text{ s.t. } D(u, v) > c \tag{5}$$

Feasibility for a target clock $c$ is checked by applying the Bellman-Ford algorithm on the constraint graph. If a negative weight cycle is detected, target $c$ is infeasible.

# 2 Software Implementation

The source code is modularized into four main files to maintain clean separation of concerns:

- **`retiming.h`:** Primary header containing definitions for `Circuit`, `RetimingResult`, `Edge`, and function prototypes.
- **`parser.c`:** Strips C-style comments (`//`) and parses parameters (`TotalNodes`, `NodeDelays`, `Edge`, `MaxClockCycle`).

- **`retiming.c`:** Core implementation containing `compute_W_and_D()`, `solve_constraints()`, and `run_retiming_algorithm()`.
- **`main.c`:** Orchestrates command-line invocation, benchmark parsing, retiming execution, and console output formatting.

# 3 Experimental Results

## 3.1 Example 1: In-Class Reference Circuit (IIR Circuit) (`example_class.txt`)

The first test case is the 4-node benchmark studied in class with initial clock period constraint $c = 3$. Node delays are $d = [1, 1, 2, 2]$.

Table 1: `example_class.txt`: Evaluated $W(u, v)$ and $D(u, v)$ Matrices

**Matrix** $W(u, v)$ **Matrix** $D(u, v)$

$$\begin{bmatrix} 0 & 1 & 1 & 2 \\ 1 & 0 & 2 & 3 \\ 1 & 0 & 0 & 3 \\ 1 & 0 & 2 & 0 \end{bmatrix} \qquad \begin{bmatrix} 1 & 4 & 3 & 3 \\ 2 & 1 & 4 & 4 \\ 4 & 3 & 2 & 6 \\ 4 & 3 & 6 & 2 \end{bmatrix}$$

**Results Summary:**

- Feasibility sequence: $c = 3$ (FEASIBLE), $c = 2$ (FEASIBLE), $c = 1$ (INFEASIBLE).
- **Optimal Clock Period ($c_{\min}$): 2**
- **Retiming Vector ($r$):** `[-1, 0, -1, -1]`
- **Retimed Edge Register Weights:** $w_r(1 \to 3) = 1$, $w_r(1 \to 4) = 2$, $w_r(2 \to 1) = 0$, $w_r(3 \to 2) = 1$, $w_r(4 \to 2) = 1$.

## 3.2 Example 2: 5-Node Pipelined Datapath (`5_node_pipeline.txt`)

A 5-node pipeline with an initial clock period constraint $c = 6$. Node delays are $d = [2, 3, 1, 4, 2]$.

Table 2: `5_node_pipeline.txt`: Evaluated $W(u, v)$ and $D(u, v)$ Matrices

**Matrix** $W(u, v)$ **Matrix** $D(u, v)$

$$\begin{bmatrix} 0 & 0 & 1 & 1 & 2 \\ 3 & 0 & 1 & 1 & 2 \\ 2 & 2 & 0 & 0 & 1 \\ 2 & 2 & 3 & 0 & 1 \\ 1 & 1 & 2 & 2 & 0 \end{bmatrix} \qquad \begin{bmatrix} 2 & 5 & 6 & 10 & 12 \\ 12 & 3 & 4 & 8 & 10 \\ 9 & 12 & 1 & 5 & 7 \\ 8 & 11 & 12 & 4 & 6 \\ 4 & 7 & 8 & 12 & 2 \end{bmatrix}$$

**Results Summary:**

- Feasibility sequence: $c = 6, 5, 4$ (FEASIBLE), $c = 3$ (INFEASIBLE due to $d(v_4) = 4$).
- **Optimal Clock Period ($c_{\min}$): 4**
- **Retiming Vector ($r$):** `[-1, 0, -1, 0, 0]`
- **Retimed Edge Register Weights:** $w_r(1 \to 2) = 1$, $w_r(2 \to 3) = 0$, $w_r(3 \to 4) = 1$, $w_r(4 \to 5) = 1$, $w_r(5 \to 1) = 0$.

### 3.3 Example 3: Multi-Loop Network (`multi_loop.txt`)

A 4-node ring topology with internal loops and initial clock constraint $c = 7$. Node delays are $d = [3, 2, 4, 1]$.

Table 3: `multi_loop.txt`: Evaluated $W(u, v)$ and $D(u, v)$ Matrices

**Matrix** $W(u, v)$ **Matrix** $D(u, v)$

$$\begin{bmatrix} 0 & 1 & 1 & 2 \\ 1 & 0 & 0 & 1 \\ 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 3 & 5 & 9 & 10 \\ 10 & 2 & 6 & 7 \\ 8 & 10 & 4 & 5 \\ 4 & 6 & 10 & 1 \end{bmatrix}$$

**Results Summary:**

- Feasibility sequence: $c = 7, 6, 5$ (FEASIBLE), $c = 4$ (INFEASIBLE).
- **Lower Bound Proof:** The directed ring $1 \to 2 \to 3 \to 4 \to 1$ has total delay $\sum d(v) = 10$ and 2 registers. The theoretical minimum clock period is $\lceil 10/2 \rceil = 5$.
- **Optimal Clock Period ($c_{\min}$): 5**
- **Retiming Vector ($r$):** `[0, -1, 0, -1]`
- **Retimed Edge Register Weights:** $w_r(1 \to 2) = 0$, $w_r(2 \to 3) = 1$, $w_r(3 \to 4) = 0$, $w_r(4 \to 1) = 1$.

## 4 Program Execution Logs

Below is the captured output from running `make test`:

```
retiming.exe example_class.txt
Loading input circuit file: example_class.txt...

=== Circuit Specifications ===
Total Nodes: 4
Node Delays: d(v1)=1 d(v2)=1 d(v3)=2 d(v4)=2
Initial Max Clock Cycle Constraint (c): 3

Executing Leiserson-Saxe Retiming Step-by-Step...
  Testing feasibility for target clock period c = 3: FEASIBLE
  Testing feasibility for target clock period c = 2: FEASIBLE
  Testing feasibility for target clock period c = 1: INFEASIBLE

=========================================================
           RETIMING RESULTS (OPTIMAL CLOCK)
=========================================================
 Minimum Feasible Clock Period (c_min): 2
 Retiming Vector r(v):
   Node 1: r = -1
   Node 2: r =  0
   Node 3: r = -1
   Node 4: r = -1
=========================================================

retiming.exe 5_node_pipeline.txt
  Testing feasibility for target clock period c = 6: FEASIBLE
  Testing feasibility for target clock period c = 5: FEASIBLE
  Testing feasibility for target clock period c = 4: FEASIBLE
```

```
  Testing feasibility for target clock period c = 3: INFEASIBLE
 Minimum Feasible Clock Period (c_min): 4
 Retiming Vector r(v): Node 1: -1, Node 2: 0, Node 3: -1, Node 4: 0, Node 5: 0

retiming.exe multi_loop.txt
  Testing feasibility for target clock period c = 7: FEASIBLE
  Testing feasibility for target clock period c = 6: FEASIBLE
  Testing feasibility for target clock period c = 5: FEASIBLE
  Testing feasibility for target clock period c = 4: INFEASIBLE
 Minimum Feasible Clock Period (c_min): 5
 Retiming Vector r(v): Node 1: 0, Node 2: -1, Node 3: 0, Node 4: -1
```

Listing 1: Console Output Log from 'make test' Execution

## 5 AI Usage and Steps

I used ClaudeAI to verify and optimize my code and structure, as I will show and discuss below. I decided to use ClaudeAI because of how efficient it is at coding and debugging code.

I didn't go with any other tool for this project, although chatGPT and Gemini are both very solid, same with MistralAI and other general purpose LLMs. Grok, Meta, and other tools didn't seem applicable due to them being built for image generation, entertainment, etc.

I also used Grammarly to help me fix any punctuation and formatting errors for this report.

ClaudeAI Workflow and Chat

Here is the code that ClaudeAI wrote to verify my code and my tests:

```c
/* Independent verifier for the Leiserson-Saxe retiming implementation.
 *
 * Links against retiming.c and parser.c but shares no algorithm with them:
 *   1. W/D oracle   : enumerate every simple path (DFS).
 *   2. Period oracle: brute-force every retiming vector in [-R, R]^(n-1).
 *   3. r validator  : retimed weights >= 0 and measured period <= c_min.
 *
 * Build: gcc -Wall -Wextra -O2 -std=c11 verify.c retiming.c parser.c -o verify
 * Run  : ./verify [num_random_tests] [seed]  [circuit files...]
 *
 * Results go to stderr because the algorithm's own progress output on
 * stdout is silenced during testing.
 */

#include "retiming.h"
#include <time.h>

#ifdef _WIN32
#define NULL_DEVICE "NUL"
#else
#define NULL_DEVICE "/dev/null"
#endif

#define BRUTE_RANGE 4   /* r(v) searched in [-4, 4]; v1 fixed to 0 */
#define BRUTE_MAX_N 6   /* brute force limited to small circuits   */

/* ---------- independent clock-period measurement ---------- */

/* Returns clock period of circuit retimed by r (r[1..n]).
 * -1: some retimed edge weight is negative (illegal retiming)
 * -2: zero-weight cycle (combinational loop)                    */
static int period_of(const Circuit *c, const int *r) {
    int n = c->num_nodes;
    int indeg[MAX_NODES + 1] = {0};
    int arr[MAX_NODES + 1];
    int queue[MAX_NODES + 1];
    int head = 0, tail = 0, done = 0, best = 0;

    for (int i = 0; i < c->edge_count; i++) {
        int wr = c->edges[i].weight + r[c->edges[i].v] - r[c->edges[i].u];
        if (wr < 0) return -1;
        if (wr == 0) indeg[c->edges[i].v]++;
    }
    for (int i = 1; i <= n; i++) {
        arr[i] = c->delay[i];
        if (indeg[i] == 0) queue[tail++] = i;
    }
    while (head < tail) {
        int u = queue[head++];
        done++;
        if (arr[u] > best) best = arr[u];
        for (int i = 0; i < c->edge_count; i++) {
            int v = c->edges[i].v;
            if (c->edges[i].u != u) continue;
            if (c->edges[i].weight + r[v] - r[u] != 0) continue;
            if (arr[u] + c->delay[v] > arr[v]) arr[v] = arr[u] + c->delay[v];
            if (--indeg[v] == 0) queue[tail++] = v;
        }
    }
    return (done < n) ? -2 : best;
}

```

```c
/* ---------- brute-force optimal period ---------- */

static int bf_best;
static void bf_rec(const Circuit *c, int *r, int idx) {
    if (idx > c->num_nodes) {
        int p = period_of(c, r);
        if (p >= 0 && (bf_best < 0 || p < bf_best)) bf_best = p;
        return;
    }
    for (int x = -BRUTE_RANGE; x <= BRUTE_RANGE; x++) {
        r[idx] = x;
        bf_rec(c, r, idx + 1);
    }
}

static int brute_force_min_period(const Circuit *c) {
    int r[MAX_NODES + 1] = {0};
    bf_best = -1;
    r[1] = 0;               /* uniform shifts do not change anything */
    bf_rec(c, r, 2);
    return bf_best;
}

/* ---------- brute-force W and D via simple-path enumeration ---------- */

static int pW[MAX_NODES + 1][MAX_NODES + 1];
static int pD[MAX_NODES + 1][MAX_NODES + 1];

static void path_dfs(const Circuit *c, int src, int cur, int wsum, int dsum,
                     int *visited) {
    /* dsum already includes delay of cur */
    if (wsum < pW[src][cur] || (wsum == pW[src][cur] && dsum > pD[src][cur])) {
        pW[src][cur] = wsum;
        pD[src][cur] = dsum;
    }
    for (int i = 0; i < c->edge_count; i++) {
        if (c->edges[i].u != cur) continue;
        int v = c->edges[i].v;
        if (visited[v]) continue;
        visited[v] = 1;
        path_dfs(c, src, v, wsum + c->edges[i].weight, dsum + c->delay[v],
    visited);
        visited[v] = 0;
    }
}

static int check_WD(const Circuit *c, const RetimingResult *res) {
    int n = c->num_nodes, bad = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) { pW[i][j] = INF; pD[i][j] = 0; }
    for (int s = 1; s <= n; s++) {
        int visited[MAX_NODES + 1] = {0};
        visited[s] = 1;
        path_dfs(c, s, s, 0, c->delay[s], visited);
    }
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) {
            if (res->W[i][j] != pW[i][j] || res->D[i][j] != pD[i][j]) {
                fprintf(stderr, "    W/D mismatch at (%d,%d): program W=%d D=%d
    , oracle W=%d D=%d\n",
                        i, j, res->W[i][j], res->D[i][j], pW[i][j], pD[i][j]);
                bad = 1;
            }
```

```
        }
    return !bad;
}

/* ---------- check one circuit; returns 1 if all checks pass ---------- */

static int verify_circuit(Circuit *c, const char *label, int verbose) {
    static RetimingResult res;
    int n = c->num_nodes;
    int ok = 1;
    int r0[MAX_NODES + 1] = {0};
    int orig = period_of(c, r0);

    if (orig == -2) {
        /* Combinational loop: program must report infeasible */
        run_retiming_algorithm(c, &res);
        if (res.is_feasible) {
            fprintf(stderr, "[FAIL] %s: combinational loop but program claims feasible\n", label);
            return 0;
        }
        if (verbose) fprintf(stderr, "[PASS] %s: combinational loop correctly rejected\n", label);
        return 1;
    }

    run_retiming_algorithm(c, &res);

    if (n <= BRUTE_MAX_N && !check_WD(c, &res)) {
        fprintf(stderr, "[FAIL] %s: W/D matrices differ from path-enumeration oracle\n", label);
        ok = 0;
    }

    /* The program only searches c <= max_clock_cycle. If that bound is below
     * the true optimum it should say infeasible; if >= optimum it must find it. */
    int opt = (n <= BRUTE_MAX_N) ? brute_force_min_period(c) : -1;

    if (!res.is_feasible) {
        if (opt >= 0 && opt <= c->max_clock_cycle) {
            fprintf(stderr, "[FAIL] %s: program says infeasible for c<=%d but optimum is %d\n",
                    label, c->max_clock_cycle, opt);
            ok = 0;
        } else if (verbose) {
            fprintf(stderr, "       %s: infeasible for the given bound (true optimum %d)\n", label, opt);
        }
        if (ok && verbose) fprintf(stderr, "[PASS] %s\n", label);
        return ok;
    }

    int ach = period_of(c, res.r);
    if (ach < 0) {
        fprintf(stderr, "[FAIL] %s: returned r is illegal (code %d)\n", label, ach);
        ok = 0;
    } else if (ach > res.min_feasible_c) {
        fprintf(stderr, "[FAIL] %s: r achieves period %d, program claims %d\n",
                label, ach, res.min_feasible_c);
        ok = 0;
    }
```

```
    if (opt >= 0 && opt != res.min_feasible_c) {
        fprintf(stderr, "[FAIL] %s: program c_min=%d but brute-force optimum=%d\n",
                label, res.min_feasible_c, opt);
        ok = 0;
    }
    if (ok && verbose)
        fprintf(stderr, "[PASS] %s: original=%d  c_min=%d  achieved=%d  brute-force optimum=%d\n",
                label, orig, res.min_feasible_c, ach, opt);
    return ok;
}

/* ---------- random circuit generation ---------- */

static void random_circuit(Circuit *c) {
    int n = 2 + rand() % 4;                  /* 2..5 nodes */
    init_circuit(c, n);
    for (int i = 1; i <= n; i++) c->delay[i] = 1 + rand() % 5;
    int m = n + rand() % (n + 1);            /* n..2n edges */
    for (int i = 0; i < m; i++)
        add_edge(c, 1 + rand() % n, 1 + rand() % n, rand() % 3);
    int r0[MAX_NODES + 1] = {0};
    int p = period_of(c, r0);
    c->max_clock_cycle = (p > 0) ? p : 10;  /* start search at original period */
}

int main(int argc, char *argv[]) {
    int num_random = 2000;
    unsigned seed = 12345;
    int first_file = 1;

    if (argc > 1 && sscanf(argv[1], "%d", &num_random) == 1) first_file = 2;
    if (argc > 2 && sscanf(argv[2], "%u", &seed) == 1) first_file = 3;

    static Circuit c;
    int fails = 0, total = 0;

    /* Silence the algorithm's own progress prints on stdout */
    if (!freopen(NULL_DEVICE, "w", stdout)) {
        fprintf(stderr, "warning: could not silence stdout\n");
    }

    fprintf(stderr, "=== Circuit files ===\n");
    const char *defaults[] = {"example_class.txt", "5_node_pipeline.txt", "multi_loop.txt"};
    int nfiles = argc - first_file;
    for (int i = 0; i < (nfiles > 0 ? nfiles : 3); i++) {
        const char *f = (nfiles > 0) ? argv[first_file + i] : defaults[i];
        if (!parse_circuit_file(f, &c)) {
            fprintf(stderr, "[FAIL] could not parse %s\n", f);
            fails++; total++;
            continue;
        }
        total++;
        if (!verify_circuit(&c, f, 1)) fails++;
    }

    /* One explicit combinational-loop case (program must reject it) */
    init_circuit(&c, 2);
    c.delay[1] = 1; c.delay[2] = 1; c.max_clock_cycle = 5;
    add_edge(&c, 1, 2, 0); add_edge(&c, 2, 1, 0);
```

```
    total++;
    if (!verify_circuit(&c, "combinational loop (expected error above)", 1)) fails++;

    fprintf(stderr, "\n=== Random circuits (valid circuits only, n=2..5, seed=%u) ===\n", seed);
    srand(seed);
    int rtotal = 0, rfail = 0;
    for (int t = 0; t < num_random; t++) {
        int r0[MAX_NODES + 1] = {0};
        do { random_circuit(&c); } while (period_of(&c, r0) == -2);
        char label[32];
        snprintf(label, sizeof label, "random #%d", t);
        rtotal++;
        if (!verify_circuit(&c, label, 0)) rfail++;
    }
    fprintf(stderr, "%d random circuits, %d failures\n", rtotal, rfail);

    fails += rfail; total += rtotal;
    fprintf(stderr, "\n=== SUMMARY: %d/%d checks passed ===\n", total - fails, total);
    return fails ? 1 : 0;
}
```

Listing 2: Script from source

# 6 Readings

## 6.1 Retiming: Theory and Practice - N. Shenoy

### Problem Addressed and Motivation

Sequential circuit performance is heavily constrained by critical path delays between memory elements (flip-flops or latches). As circuit operating frequencies increase, combinational logic delay between registers becomes the primary bottleneck. This paper surveys the foundational theory and practical applications of retiming—a structural CAD optimization technique that relocates registers across logic gates without altering circuit functionality. The primary motivation is to automate clock period minimization, register count minimization (area optimization), and power reduction during logic synthesis and physical design.

### Key Concepts and Assumptions

Synchronous Circuit Graph ($G = (V, E, d, w)$): Vertices $V$ represent combinational gates with non-negative delays $d(v)$, and directed edges $E$ represent interconnect wires with non-negative register weights $w(e)$. Preservation of Functionality: Retiming preserves the latency and state behavior of the system, provided register counts along all directed cycles remain invariant. Host Node Assumption: Pipelined inputs/outputs are connected to a pseudo "host" node $v_0$ to preserve peripheral register counts and external latency bounds. Fixed Gate Delays: Gate propagation delays are assumed fixed and independent of register placement (ignoring detailed interconnect RC loads during early retiming).

### Main Approach and Core Algorithms

The paper categorizes retiming into two primary formulations: Clock Period Minimization (OPT1): Determines a retiming vector $r(v) \in \mathbb{Z}$ that minimizes clock cycle period $c$. Computes the $W(u, v)$ matrix (minimum registers along paths $u \rightsquigarrow v$) via Floyd-Warshall. Computes the $D(u, v)$ matrix (maximum zero-register propagation delay along critical paths). Solves difference constraints using Bellman-Ford shortest-path algorithms.

Area Minimization under Clock Constraint (OPT2): Formulates register count minimization under a fixed clock period constraint as a Dual Network Flow (Minimum Cost Flow) problem.

### Technical Equation / Algorithm Explanation

The retiming transformation alters edge register counts according to:

$$w_r(e) = w(e) + r(v) - r(u) \quad \forall e = (u, v) \in E$$

Legality Constraint: $w_r(e) \geq 0 \implies r(u) - r(v) \leq w(e)$ ensures no edge has negative registers. Clock Constraint: For any pair $(u, v)$ where $D(u, v) > c$, at least one register must separate $u$ and $v$:

$$w_r(p) \geq 1 \implies r(u) - r(v) \leq W(u, v) - 1$$

These linear inequalities form a system of difference constraints solvable in $O(|V| \cdot |E|)$ time.

### Explanation of Important Figures and Examples

*Figure 1 (Correlator Circuit Example)*: Demonstrates how shifting registers backward across a heavy adder block reduces critical path delay from $D_{\max} = 7$ down to $c_{\min} = 3$ without adding extra registers. Constraint Graph Construction: Illustrates how inequalities are mapped into directed edges where negative cycle checks evaluate clock period feasibility.

### Experimental Results and Conclusions

*Clock Speedup*: Experimental evaluations on ISCAS benchmarks demonstrate clock frequency improvements ranging from 15% to 45%.

*Area Penalty*: Pure clock minimization (OPT1) can increase total flip-flop count; however, constrained area optimization (OPT2) successfully recovers area while maintaining the optimal clock period.

### Major Contributions and Limitations

Contributions: Unifies retiming theory across gate-level design, FPGA synthesis, and high-level scheduling. Establishes polynomial-time dual network flow algorithms for register minimization. Limitations: Classical retiming assumes static gate delays, failing to capture post-layout interconnect delays dominated by deep-submicron wire resistance/capacitance.

### Personal Assessment

The most interesting aspect to me is the mapping of a hardware timing problem into a pure graph-theoretic shortest path/network flow problem is elegant.

The most difficult aspect was understanding dual network flow transformations for register minimization with non-unit node fanout costs.

## 6.2 Retiming for Wire Pipelining in System-on-Chip - H. Zhou

### Problem Addressed and Motivation

In modern System-on-Chip (SoC) designs fabricated in deep-submicron processes, interconnect wire delays dominate total logic delay. Long global wires connecting IP blocks often require multiple clock cycles for signal propagation.

Traditional retiming assumes registers are moved over zero-delay wires between logic gates. Zhou addresses the problem of wire pipelining: placing registers directly on long interconnect wires under strict routing and latency constraints to maintain high clock frequencies in SoCs.

### Key Concepts and Assumptions

Wire Delay Modeling:
Wire delay is proportional to physical wire length rather than being negligible.

Keepout Zones & Floorplan Constraints:
Registers cannot be arbitrarily placed anywhere on a chip; IP blocks and macro boundaries create spatial "keepout zones" where registers are forbidden.

Sequential Equivalence:
Global wire pipelining must not introduce deadlock or timing race conditions across multi-clock IP block interfaces.

### Main Approach and Core Algorithms

Zhou formulates wire pipelining as a Regular Retiming Extension under physical layout constraints:
Models global wires as segmented edges with inserted pseudo-nodes representing potential register insertion sites.

Formulates upper and lower bounds on relative register positions along physical routing paths.

Proposes an efficient polynomial-time algorithm based on incremental shortest-path updates on a modified constraint graph that incorporates spatial bounds.

### Technical Equation / Algorithm Explanation

To account for wire delay $d(e)$ alongside gate delay $d(v)$, the maximum combinational path delay condition includes wire segment lengths:

$$D_{\text{wire}}(p) = \sum_{v \in p} d(v) + \sum_{e \in p} d(e) \leq c$$

When a wire $e = (u, v)$ has physical length $L(e)$ exceeding clock distance limit $W_{\max}$, Zhou enforces that the number of inserted pipeline registers $k$ satisfies:

$$\left\lceil \frac{L(e)}{W_{\max}} \right\rceil \leq w_r(e) \leq \text{Upper Bound}$$

This adds physical bounds into the $r(u) - r(v)$ difference constraint system.

**Explanation of Important Figures and Examples**

*Figure 2* (SoC Wire Segmentation  Keepout Zones): Illustrates global routing wires traversing across macro blocks. Demonstrates how pseudo-nodes placed outside IP block boundaries prevent invalid register placement inside macro blocks.

*Figure 4* (Constraint Graph Expansion): Shows how physical distance bounds map into upper and lower bound edges in the difference constraint graph.

**Experimental Results and Conclusions**

*Clock Frequency Retention*: Successfully achieves target clock frequencies in deep-submicron SoC designs where unpipelined wire delays originally caused up to 60% timing degradation.

*Algorithmic Efficiency*: The proposed algorithm scales efficiently, taking under few seconds on benchmarks with thousands of global wires.

**Major Contributions and Limitations**

*Contributions*: Bridges the gap between architectural retiming and physical layout/floorplanning. Solves wire pipelining under arbitrary spatial register insertion constraints.

*Limitations*: Requires early fixed global routing topologies; if physical routing changes during detailed placement, wire delays must be re-evaluated.

**Personal Assessment**

*Most Interesting Aspect*: Extending classic graph algorithms (like Bellman-Ford) to solve physical design layout constraints without resorting to expensive integer linear programming (ILP).

*Most Difficult Aspect*: Tracking multi-cycle latency constraints across intersecting global wire paths with non-uniform keepout zone boundaries.

# 7 Conclusion

The Leiserson-Saxe retiming engine was successfully implemented in modular C and verified. Across all test benchmarks, the algorithm correctly identified $W(u, v)$ and $D(u, v)$ matrices, formulated system difference constraints, and converged to optimal minimum clock periods ($c_{\min} = 2, 4, 5$) while preserving positive edge register counts.

The readings also were very insightful to help understand the vastness and the usage of retiming in physical designs, which was very interesting to me.

Another thing that we can see is that my answer was correct, and it was verified by ClaudeAI, as shown above.